

Advanced Guide: Building the Seller Acquisition ROI Model

This follows the exact same six-week plan as `tutorial.md` — same datasets, same milestones, same "done when" bars. It exists because `tutorial.md` tells you *what* to build each week and points at docs; this tells you *how*, with full working scripts wired to this project's actual data model (as specified in `business-case.md`). Read `tutorial.md` first for the schedule and the "why this design" reasoning — this is the implementation companion, not a replacement.

Every script below is written against the real column names and grains from `business-case.md` §B (data model) and §C (acquisition ROI model). Copy them as a starting point, adapt as your own ingestion reveals real column quirks (Kaggle CSVs always have a surprise or two — this project's particular one is that `declared_monthly_revenue`, `declared_product_catalog_size`, `has_company`, `has_gtin`, and `average_stock` on `closed_deals` are sparsely populated; check null rates before you build anything downstream of them).

Day 0 — Setup, in full

`tutorial.md` gives you the checklist. Here's the actual sequence, in order, so nothing depends on something you haven't done yet.

```
# 1. Auth and project
gcloud auth login
gcloud projects create pnl-forecast --name="PNL Forecast"
# ^ GCP will silently suffix this to something like pnl-forecast-507416 if
the short
# name's taken globally — that's normal, not an error. Use whatever it
actually
# creates consistently from here on; `gcloud config get-value project` is
the source
# of truth, not the name you typed.
gcloud config set project pnl-forecast # or the suffixed name it actually
gave you
gcloud auth application-default login # separate from `gcloud auth login`
— this is what

# client libraries (google-cloud-
storage, dbt-bigquery)

# actually read. Skipping this is
the #1 cause of

# "works with bq CLI but not in
my script."

# 2. Enable the APIs you'll actually call
gcloud services enable bigquery.googleapis.com storage.googleapis.com

# 3. Budget alert — do this before any query, not after
gcloud billing budgets create \
```

```

--billing-account=$(gcloud billing accounts list --
format='value(ACCOUNT_ID)' --limit=1) \
--display-name="pnl-forecast-5eur" \
--budget-amount=5EUR \
--threshold-rule=percent=0.5 \
--threshold-rule=percent=1.0

# 4. Bucket, region matters (pick one region and stay consistent – cross-
region
#   BigQuery <-> GCS access has its own cost and latency implications)
gcloud storage buckets create gs://profit-forecast-data --location=EU
# ^ Yes, the bucket name still says "profit-forecast" even though the
project and the
#   business case have both moved on. That's deliberate: this bucket
predates the current
#   naming, GCS bucket names can't be changed in place (only create-copy-
delete, not worth
#   it for a dev bucket), and pretending otherwise in these docs would just
make the real
#   `gcloud storage buckets list` output look wrong next to what you read
here. If you're
#   starting completely fresh, there's nothing stopping you from naming
yours
#   `pnl-forecast-data` instead – just update every `gs://` reference below
to match.

# 5. Python project
uv init --python 3.14
uv add duckdb google-cloud-storage google-cloud-bigquery pandas pyarrow
kaggle
uv add --dev pytest ruff

# 6. Kaggle credentials
# Download kaggle.json from https://www.kaggle.com/settings/account, then:
mkdir -p ~/.kaggle && mv ~/Downloads/kaggle.json ~/.kaggle/kaggle.json
chmod 600 ~/.kaggle/kaggle.json

```

Done when:

```

bq ls                                # empty list, no error
duckdb -c "select 1"                 # returns 1
kaggle datasets list -s olist        # returns a table of Olist datasets
gcloud auth application-default print-access-token # prints a token, not
an error

```

Neither **brazilian-ecommerce** nor **marketing-funnel-olist** is a Kaggle *competition* — they're public datasets, so there's no rules-acceptance click-through and no 403 to debug here. That's a genuine simplification over the previous version of this project, which needed a "did you accept the competition rules on the web page yet?" troubleshooting note at this exact spot. **kaggle datasets download** on a public dataset just works once your API key is in place.

Week 1 — Ingestion and Python hygiene

1.1 Why raw stays raw

The staging/intermediate/marts split (used from Week 3 onward) only works if **raw** never lies about what the source actually contained. Any cleaning, renaming, or type coercion done at ingestion time is invisible later — if **ingest.py** silently drops nulls, nobody debugging a dbt model three weeks from now will think to look in a Python script for the cause. Ingestion's job is narrow: get bytes from Kaggle to GCS as Parquet, unchanged in content, changed only in format. That matters especially here: **closed_deals**' sparsely-populated declared fields need to survive ingestion exactly as sparse as they arrived, so that the *staging* layer — not this one — is where you decide how to handle the nulls, on the record, in SQL someone can read.

1.2 **download.py**

```
#!/usr/bin/env python3
"""download.py — fetch Olist's marketing funnel and e-commerce datasets
from Kaggle
to a local raw/ directory.

Requires ~/.kaggle/kaggle.json (see Day 0). Idempotent: re-running skips
files
already present unless --force is passed.

Usage:
    python download.py --dest data/raw
    python download.py --dest data/raw --force
"""
from __future__ import annotations

import argparse
import logging
import zipfile
from pathlib import Path

from kaggle.api.kaggle_api_extended import KaggleApi

logger = logging.getLogger("download")

# (kaggle dataset slug, local subdir) — both are Kaggle *datasets*, not
# competitions,
# so there's a single download path with no rules-acceptance branch to
# handle.
SOURCES = [
    ("olistbr/brazilian-ecommerce", "olist_ecom"),
    ("olistbr/marketing-funnel-olist", "olist_funnel"),
]

def parse_args(argv: list[str] | None = None) -> argparse.Namespace:
    p = argparse.ArgumentParser(description="Download Olist datasets for
```

```

the acquisition ROI model.")
    p.add_argument("--dest", type=Path, default=Path("data/raw"))
    p.add_argument("--force", action="store_true", help="Re-download even
if already present")
    p.add_argument("-v", "--verbose", action="store_true")
    return p.parse_args(argv)

def download_one(api: KaggleApi, identifier: str, dest: Path, force: bool)
-> None:
    marker = dest / ".downloaded"
    if marker.exists() and not force:
        logger.info("Skipping %s (already downloaded, use --force to
redo)", identifier)
        return

    dest.mkdir(parents=True, exist_ok=True)
    logger.info("Downloading %s -> %s", identifier, dest)
    api.dataset_download_files(identifier, path=str(dest), quiet=False)

    for zf in dest.glob("*.zip"):
        logger.debug("Extracting %s", zf)
        with zipfile.ZipFile(zf) as z:
            z.extractall(dest)
        zf.unlink()

    marker.touch()

def main(argv: list[str] | None = None) -> int:
    args = parse_args(argv)
    logging.basicConfig(
        level=logging.DEBUG if args.verbose else logging.INFO,
        format="%(asctime)s %(levelname)-8s %(message)s",
    )
    api = KaggleApi()
    api.authenticate()

    for identifier, subdir in SOURCES:
        download_one(api, identifier, args.dest / subdir, args.force)

    logger.info("Done. Raw data in %s", args.dest)
    return 0

if __name__ == "__main__":
    import sys

    sys.exit(main())

```

Why a `.downloaded` marker file rather than checking for the CSVs directly: the extracted filenames vary by source and you don't want `download.py` to need per-source knowledge of what files to expect — that's

`ingest.py`'s job. (This is also the one bug worth watching for if you're comparing against an earlier draft of this script: a missing `if __name__ == "__main__":` guard means `main()` never actually runs when you invoke the file directly — `python download.py` would silently do nothing and exit 0. Keep that guard.)

1.3 `ingest.py` — the DuckDB streaming conversion

Nothing in this project is 125M rows the way the previous version's Favorita table was — the largest table here, `geolocation`, is 1,000,163 rows of five columns (~61 MB as CSV). DuckDB's `COPY` still beats pandas for this, though, and the habit is worth building on data this size rather than for the first time on data that would actually punish you for skipping it:

```
#!/usr/bin/env python3
"""ingest.py — convert raw Kaggle CSVs to partitioned Parquet in GCS, via
DuckDB.

DuckDB streams the CSV -> Parquet conversion without loading full files
into
memory, and can write straight to gs:// if the httpfs extension is loaded
with
credentials — but writing locally then `gcloud storage cp -r` is simpler to
debug on a first pass, so that's what this does. Switch to direct GCS
writes
once you trust the pipeline.

Usage:
    python ingest.py --raw data/raw --out data/parquet --bucket profit-
forecast-data
"""
from __future__ import annotations

import argparse
import logging
import subprocess
from pathlib import Path

import duckdb

logger = logging.getLogger("ingest")

# Each entry: source CSV (relative to --raw), output table name, partition
columns (or None)
ECOM_TABLES = [
    ("olist_ecom/olist_orders_dataset.csv", "olist_orders", ["year",
"month"]),
    ("olist_ecom/olist_order_items_dataset.csv", "olist_order_items",
None),
    ("olist_ecom/olist_order_payments_dataset.csv", "olist_order_payments",
None),
    ("olist_ecom/olist_order_reviews_dataset.csv", "olist_order_reviews",
None),
    ("olist_ecom/olist_customers_dataset.csv", "olist_customers", None),
    ("olist_ecom/olist_sellers_dataset.csv", "olist_sellers", None),
```

```

    ("olist_ecom/olist_products_dataset.csv", "olist_products", None),
    # 1,000,163 rows – the one table with real row-count heft. Row count
    isn't the same
    # axis as byte volume, though: five float/string columns still lands at
    ~61 MB, well
    # inside "unpartitioned is fine" territory. Kept unpartitioned
    deliberately, as a
    # reminder that "big row count" and "worth partitioning" aren't the
    same question.
    ("olist_ecom/olist_geolocation_dataset.csv", "olist_geolocation",
None),
    ("olist_ecom/product_category_name_translation.csv",
"product_category_name_translation", None),
]

FUNNEL_TABLES = [
    ("olist_funnel/olist_marketing_qualified_leads_dataset.csv",
"olist_mql", None),
    ("olist_funnel/olist_closed_deals_dataset.csv", "olist_closed_deals",
None),
]

def parse_args(argv: list[str] | None = None) -> argparse.Namespace:
    p = argparse.ArgumentParser(description="Convert raw CSVs to
partitioned Parquet.")
    p.add_argument("--raw", type=Path, default=Path("data/raw"))
    p.add_argument("--out", type=Path, default=Path("data/parquet"))
    p.add_argument("--bucket", required=True, help="GCS bucket to sync to
(no gs:// prefix)")
    p.add_argument("--skip-upload", action="store_true", help="Convert
locally only, don't sync")
    p.add_argument("-v", "--verbose", action="store_true")
    return p.parse_args(argv)

def convert_table(
    con: duckdb.DuckDBPyConnection,
    csv_path: Path,
    table_name: str,
    out_dir: Path,
    partition_by: list[str] | None,
) -> None:
    if not csv_path.exists():
        logger.warning("Missing %s, skipping %s", csv_path, table_name)
        return

    table_out = out_dir / table_name
    table_out.mkdir(parents=True, exist_ok=True)

    if table_name == "olist_orders":
        # The one table that gets a real date split, for the partition/cost
        lessons in
        # Week 2. order_purchase_timestamp is a full timestamp, not a bare

```

```

date.
    query = f"""
        COPY (
            SELECT *,
                EXTRACT(year FROM order_purchase_timestamp) AS year,
                EXTRACT(month FROM order_purchase_timestamp) AS
month
                    FROM read_csv_auto('{csv_path}', header=true)
                ) TO '{table_out}' (FORMAT PARQUET, PARTITION_BY (year, month),
OVERWRITE_OR_IGNORE true)
        """
    else:
        query = f"""
            COPY (SELECT * FROM read_csv_auto('{csv_path}', header=true))
            TO '{table_out / (table_name + ".parquet")}' (FORMAT PARQUET)
            """

    logger.info("Converting %s -> %s", csv_path, table_out)
    con.execute(query)

def main(argv: list[str] | None = None) -> int:
    args = parse_args(argv)
    logging.basicConfig(
        level=logging.DEBUG if args.verbose else logging.INFO,
        format="%(asctime)s %(levelname)-8s %(message)s",
    )

    con = duckdb.connect() # in-memory – we're only using DuckDB as a
conversion engine here
    args.out.mkdir(parents=True, exist_ok=True)

    all_tables = [*ECOM_TABLES, *FUNNEL_TABLES]
    for rel_path, table_name, partition_by in all_tables:
        convert_table(con, args.raw / rel_path, table_name, args.out,
partition_by)

    if not args.skip_upload:
        for prefix, tables in [("raw/olist_ecom", ECOM_TABLES),
("raw/olist_funnel", FUNNEL_TABLES)]:
            for _, table_name, _ in tables:
                local = args.out / table_name
                if not local.exists():
                    continue
                dest = f"gs://{args.bucket}/{prefix}/{table_name}/"
                logger.info("Syncing %s -> %s", local, dest)
                subprocess.run(
                    ["gcloud", "storage", "rsync", "-r", str(local), dest],
                    check=True,
                )

    logger.info("Done.")
    return 0

```

```
if __name__ == "__main__":
    import sys

    sys.exit(main())
```

Why `gcloud storage rsync` and not a single `cp -r`: `rsync` is idempotent and incremental — running `ingest.py` twice after a partial failure only uploads what changed, and it's what makes the "running twice leaves the bucket unchanged" done-bar in `tutorial.md` actually true, rather than something you have to remember to check by hand.

1.4 The one `pytest` test `tutorial.md` asks for

```
# test_ingest.py
from pathlib import Path
import ingest

def test_orders_partitioned_by_year_month(tmp_path: Path):
    csv = tmp_path / "olist_orders_dataset.csv"
    csv.write_text(
        "order_id,customer_id,order_status,order_purchase_timestamp,"
        "order_approved_at,order_delivered_carrier_date,order_delivered_customer_da"
        "te,"
        "order_estimated_delivery_date\n"
        "abc123,cust1,delivered,2017-07-15 10:00:00,,,,\n"
    )

    import duckdb
    con = duckdb.connect()
    out_dir = tmp_path / "out"
    ingest.convert_table(con, csv, "olist_orders", out_dir, ["year",
"month"])

    written = list((out_dir / "olist_orders").rglob("*.parquet"))
    assert any("year=2017" in str(p) and "month=7" in str(p) for p in
written)
```

Be able to explain: why `raw` keeps source shape and does no cleaning (§1.1 above), and why columnar Parquet beats CSV even at this modest a scale — columnar storage stores each column contiguously and compresses within it, and a CSV's repeated text encoding of every value on every line is wasteful independent of how many rows there are.

Week 2 — Two engines, one dataset

2.1 `load.py` — BigQuery native tables

```
#!/usr/bin/env python3
"""load.py — load Parquet from GCS into partitioned/clustered BigQuery
tables.

Usage:
    python load.py --bucket profit-forecast-data --dataset raw --project
    pnl-forecast
"""
from __future__ import annotations

import argparse
import logging

from google.cloud import bigquery

logger = logging.getLogger("load")

# table_name -> (partition_field, cluster_fields) — None means no
# partitioning/clustering.
# olist_orders is the only table that earns it; everything else is small
# enough that
# partitioning would add overhead without adding pruning benefit.
TABLE_CONFIG = {
    "olist_orders": ("order_purchase_timestamp", ["order_status"]),
    "olist_order_items": (None, None),
    "olist_order_payments": (None, None),
    "olist_order_reviews": (None, None),
    "olist_customers": (None, None),
    "olist_sellers": (None, None),
    "olist_products": (None, None),
    "olist_geolocation": (None, None),
    "product_category_name_translation": (None, None),
    "olist_mql": (None, None),
    "olist_closed_deals": (None, None),
}

ECOM_TABLE_NAMES = {
    "olist_orders", "olist_order_items", "olist_order_payments",
    "olist_order_reviews",
    "olist_customers", "olist_sellers", "olist_products",
    "olist_geolocation",
    "product_category_name_translation",
}

def parse_args(argv: list[str] | None = None) -> argparse.Namespace:
    p = argparse.ArgumentParser(description="Load GCS Parquet into
    BigQuery.")
    p.add_argument("--bucket", required=True)
    p.add_argument("--dataset", default="raw")
    p.add_argument("--project", required=True)
    p.add_argument("--location", default="EU")
```

```

p.add_argument("-v", "--verbose", action="store_true")
return p.parse_args(argv)

def load_table(
    client: bigquery.Client,
    dataset: str,
    table_name: str,
    source_uri: str,
    partition_field: str | None,
    cluster_fields: list[str] | None,
) -> None:
    table_ref = f"{client.project}.{dataset}.{table_name}"

    job_config = bigquery.LoadJobConfig(
        source_format=bigquery.SourceFormat.PARQUET,
        write_disposition=bigquery.WriteDisposition.WRITE_TRUNCATE, #
replace, not append -
        # matches the "reloading a month replaces rather than duplicates"
done-bar
    )
    if partition_field:
        job_config.time_partitioning =
bigquery.TimePartitioning(field=partition_field)
    if cluster_fields:
        job_config.clustering_fields = cluster_fields

    logger.info("Loading %s -> %s (partition=%s, cluster=%s)",
                source_uri, table_ref, partition_field, cluster_fields)
    job = client.load_table_from_uri(source_uri, table_ref,
job_config=job_config)
    job.result() # blocks until done, raises on failure

    table = client.get_table(table_ref)
    logger.info("Loaded %s: %d rows, %.2f MB", table_ref, table.num_rows,
                table.num_bytes / 1e6)

def main(argv: list[str] | None = None) -> int:
    args = parse_args(argv)
    logging.basicConfig(
        level=logging.DEBUG if args.verbose else logging.INFO,
        format="%(asctime)s %(levelname)-8s %(message)s",
    )

    client = bigquery.Client(project=args.project, location=args.location)
    client.create_dataset(args.dataset, exists_ok=True)

    for table_name, (partition_field, cluster_fields) in
TABLE_CONFIG.items():
        prefix = "olist_ecom" if table_name in ECOM_TABLE_NAMES else
"olist_funnel"
        source_uri =
f"gs://{args.bucket}/raw/{prefix}/{table_name}/*.parquet"

```

```

        load_table(client, args.dataset, table_name, source_uri,
                    partition_field, cluster_fields)

        logger.info("Done.")
        return 0

if __name__ == "__main__":
    import sys

    sys.exit(main())

```

Note the Hive-partitioned Parquet from `ingest.py` (`year=2017/month=7/*.parquet`) needs `source_uri_prefix` and hive partitioning options if you load it as-is; the simpler path used above assumes `olist_orders`'s partition columns get re-derived as a genuine BigQuery `TimePartitioning` on `order_purchase_timestamp` at load time rather than relying on the source file layout — which is also *more* useful, since BigQuery's own partition pruning then works regardless of how the Parquet files were laid out in GCS.

2.2 DuckDB reading the same Parquet, no load step

```

-- dev.sql — run with `duckdb data/local.duckdb < dev.sql`, or via dbt
sources
CREATE OR REPLACE VIEW olist_orders AS
    SELECT * FROM read_parquet('data/parquet/olist_orders/**/*.parquet',
hive_partitioning=true);

CREATE OR REPLACE VIEW olist_order_items AS
    SELECT * FROM read_parquet('data/parquet/olist_order_items/*.parquet');

CREATE OR REPLACE VIEW olist_sellers AS
    SELECT * FROM read_parquet('data/parquet/olist_sellers/*.parquet');

CREATE OR REPLACE VIEW olist_mql AS
    SELECT * FROM read_parquet('data/parquet/olist_mql/*.parquet');

CREATE OR REPLACE VIEW olist_closed_deals AS
    SELECT * FROM
read_parquet('data/parquet/olist_closed_deals/*.parquet');

```

No copy, no wait — a view over the Parquet files. This is why the dev loop in Appendix C.5 (of `tutorial.md`) is "full history, still seconds": there's no load job, just DuckDB's vectorized scan directly against the files on disk (or against `gs://` directly, via the `httpfs` extension, if you skip the local sync step).

2.3 The cost experiment script

`tutorial.md` asks you to record three numbers: BigQuery bytes-scanned on a partitioned table, the same on an unpartitioned copy, and DuckDB wall-clock. `business-case.md` §6 is explicit that the real bytes here

are small — the point of this script is the *ratio*, not the absolute numbers, and that ratio holds regardless of table size:

```
#!/usr/bin/env python3
"""cost_experiment.py – compare BigQuery partitioned vs unpartitioned bytes
scanned,
and DuckDB wall-clock, for the same one-month query. Writes results as
Markdown you
can paste straight into your README.

The absolute bytes-scanned numbers here will be small regardless of which
table you
point this at – see business-case.md §6 on why this project's cost
discipline is
practiced rather than felt. What should hold up is the ratio between the
two
BigQuery configs, which is the actual thing worth recording.

Usage:
    python cost_experiment.py --project pnl-forecast --duckdb
data/local.duckdb
"""
from __future__ import annotations

import argparse
import time

import duckdb
from google.cloud import bigquery

MONTH_QUERY_BQ = """
    SELECT order_status, COUNT(*) AS n_orders
    FROM `{project}.raw.{table}`
    WHERE order_purchase_timestamp BETWEEN '2017-07-01' AND '2017-07-31'
    GROUP BY order_status
"""

MONTH_QUERY_DUCKDB = """
    SELECT order_status, COUNT(*) AS n_orders
    FROM olist_orders
    WHERE order_purchase_timestamp BETWEEN '2017-07-01' AND '2017-07-31'
    GROUP BY order_status
"""

def bq_bytes_scanned(client: bigquery.Client, project: str, table: str) ->
int:
    job_config = bigquery.QueryJobConfig(dry_run=True,
use_query_cache=False)
    query = MONTH_QUERY_BQ.format(project=project, table=table)
    job = client.query(query, job_config=job_config)
    return job.total_bytes_processed
```

```

def duckdb_wall_clock(con: duckdb.DuckDBPyConnection) -> float:
    start = time.monotonic()
    con.execute(MONTH_QUERY_DUCKDB).fetchall()
    return time.monotonic() - start

def main() -> int:
    p = argparse.ArgumentParser()
    p.add_argument("--project", required=True)
    p.add_argument("--duckdb", required=True)
    p.add_argument(
        "--unpartitioned-table",
        default="olist_orders_unpartitioned",
        help="Name of the unpartitioned copy you created for this
experiment",
    )
    args = p.parse_args()

    bq = bigquery.Client(project=args.project)
    partitioned_bytes = bq_bytes_scanned(bq, args.project, "olist_orders")
    unpartitioned_bytes = bq_bytes_scanned(bq, args.project,
args.unpartitioned_table)

    con = duckdb.connect(args.duckdb)
    duckdb_seconds = duckdb_wall_clock(con)

    print("| Engine | Config | Bytes scanned / time |")
    print("|---|---|---|")
    print(f"| BigQuery | partitioned by order_purchase_timestamp |
{partitioned_bytes / 1e6:.2f} MB |")
    print(f"| BigQuery | unpartitioned | {unpartitioned_bytes / 1e6:.2f} MB
|")
    print(f"| DuckDB | local Parquet | {duckdb_seconds * 1000:.0f} ms |")
    print(f"\nPartition pruning saved {(1 - partitioned_bytes /
unpartitioned_bytes) * 100:.0f}% "
        f"of bytes scanned for this query – small absolute numbers, same
ratio you'd see "
        f"at production scale.")
    return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

Create the unpartitioned copy once, deliberately, for the comparison:

```

CREATE TABLE raw.olist_orders_unpartitioned AS
SELECT * FROM raw.olist_orders;

```

Be able to explain: partition pruning (the planner skips reading partitions outside the **WHERE** range before scanning starts, because partition metadata is checked, not row content); why **LIMIT** doesn't reduce BigQuery cost (the limit is applied to the result *after* the full column scan the query requires — the engine doesn't know it can stop early until the scan is already priced); why selecting fewer columns does (columnar storage means each unselected column is never read off disk at all — cost is bytes-of-columns-touched, not rows-touched).

Week 3 — dbt, portable across both engines

3.1 Project skeleton

```
pnl_forecast/
├─ dbt_project.yml
├─ profiles.yml
├─ models/
│   └─ staging/
│       └─ olist/
│           ├── _olist__sources.yml
│           ├── stg_olist__mql.sql
│           ├── stg_olist__closed_deals.sql
│           ├── stg_olist__orders.sql
│           ├── stg_olist__order_items.sql
│           ├── stg_olist__order_payments.sql
│           ├── stg_olist__order_reviews.sql
│           ├── stg_olist__sellers.sql
│           ├── stg_olist__products.sql
│           └─ stg_olist__customers.sql
│   └─ intermediate/
│       ├── int_olist__funnel_weekly.sql
│       ├── int_olist__seller_monthly_gmv.sql
│       └─ int_olist__seller_acquisition.sql
│   └─ marts/
│       ├── fct_channel_funnel.sql
│       ├── fct_seller_ltv.sql
│       └─ _marts__schema.yml
├─ seeds/
│   ├── channel_cost_benchmarks.csv
│   └─ scenario_parameters.csv
└─ tests/
    └─ assert_attributed_sellers_within_bound.sql
```

Both Kaggle downloads land in one source group, **olist**, because — unlike the previous version of this project — they share a real key. `_olist__sources.yml` declares one `source: olist` with all eleven tables under it.

3.2 Staging: `stg_olist__closed_deals.sql` — the sparse-column gotcha

The model `tutorial.md` calls out explicitly for Week 1: several **closed_deals** columns are mostly null. Staging is where you decide what to do about that — cast them through, don't drop them, and don't let a

downstream model silently treat a null as a zero.

```
-- models/staging/olist/stg_olist__closed_deals.sql
with source as (
    select * from {{ source('olist', 'olist_closed_deals') }}
),

renamed as (
    select
        mql_id,
        seller_id,
        sdr_id,
        sr_id,
        {{ dbt.safe_cast('won_date',
api.Column.translate_type('timestamp')) }} as won_date,
        business_segment,
        lead_type,
        business_type,
        -- Sparsely populated in the source – pass through as-is rather
than coalescing
        -- to a default. A null here means "not declared," which is a
different fact
        -- than "declared zero," and collapsing the two would be a silent
lie.
        declared_monthly_revenue,
        declared_product_catalog_size,
        has_company,
        has_gtin
    from source
    -- one row per mql_id in the raw file, but dedupe defensively rather
than assume it
    qualify row_number() over (partition by mql_id order by won_date desc)
= 1
)

select * from renamed
```

3.3 Staging: stg_olist__order_items.sql

```
-- models/staging/olist/stg_olist__order_items.sql
with source as (
    select * from {{ source('olist', 'olist_order_items') }}
),

renamed as (
    select
        order_id,
        order_item_id,
        product_id,
        seller_id,
```

```

        {{ dbt.safe_cast('shipping_limit_date',
api.Column.translate_type('timestamp')) }}
        as shipping_limit_date,
        price,
        freight_value
    from source
)

select * from renamed

```

3.4 Intermediate: `int_olist__seller_monthly_gmv.sql`

```

-- models/intermediate/int_olist__seller_monthly_gmv.sql
with order_items as (
    select * from {{ ref('stg_olist__order_items') }}
),
orders as (
    select * from {{ ref('stg_olist__orders') }}
),
reviews as (
    select * from {{ ref('stg_olist__order_reviews') }}
),
joined as (
    select
        order_items.seller_id,
        {{ dbt.date_trunc('month', 'orders.order_purchase_timestamp') }} as
month_start,
        order_items.order_id,
        order_items.price,
        order_items.freight_value,
        reviews.review_score
    from order_items
    inner join orders on order_items.order_id = orders.order_id
    left join reviews on order_items.order_id = reviews.order_id
)

select
    seller_id,
    month_start,
    sum(price) as gmv,
    sum(freight_value) as total_freight,
    count(distinct order_id) as n_orders,
    avg(review_score) as avg_review_score
from joined
group by 1, 2

```

This is the outcome spine every later mart joins onto. It doesn't know or care whether a seller has an acquisition record — that's the point.

3.5 Intermediate: `int_olist__funnel_weekly.sql`

The gotcha `tutorial.md` calls out: `origin` lives on the MQL table, not on `closed_deals`. A closed-deal row has to join back to its own MQL record to inherit a channel at all.

```
-- models/intermediate/int_olist__funnel_weekly.sql
with mql as (
  select * from {{ ref('stg_olist__mql') }}
),

closed_deals as (
  select * from {{ ref('stg_olist__closed_deals') }}
),

mql_weekly as (
  select
    origin,
    {{ dbt.date_trunc('week', 'first_contact_date') }} as week_start,
    count(*) as n_leads
  from mql
  group by 1, 2
),

-- inherit origin via the mql join – closed_deals has no origin column of
its own
closed_with_origin as (
  select
    mql.origin,
    {{ dbt.date_trunc('week', 'closed_deals.won_date') }} as
week_start,
    closed_deals.seller_id
  from closed_deals
  inner join mql on closed_deals.mql_id = mql.mql_id
),

closed_weekly as (
  select
    origin,
    week_start,
    count(*) as n_closed
  from closed_with_origin
  group by 1, 2
)

select
  coalesce(mql_weekly.origin, closed_weekly.origin) as origin,
  coalesce(mql_weekly.week_start, closed_weekly.week_start) as
week_start,
  coalesce(mql_weekly.n_leads, 0) as n_leads,
```

```

    coalesce(closed_weekly.n_closed, 0) as n_closed
from mql_weekly
full outer join closed_weekly
  on mql_weekly.origin = closed_weekly.origin
  and mql_weekly.week_start = closed_weekly.week_start

```

The full outer join matters: a week can have closes with no new leads that same week (leads worked over from a prior week), and vice versa. An inner join would silently drop either side.

3.6 Intermediate: `int_olist__seller_acquisition.sql`

```

-- models/intermediate/int_olist__seller_acquisition.sql
with closed_deals as (
  select * from {{ ref('stg_olist__closed_deals') }}
),
mql as (
  select * from {{ ref('stg_olist__mql') }}
)

select
  closed_deals.seller_id,
  mql.origin,
  closed_deals.lead_type,
  closed_deals.business_type,
  closed_deals.business_segment,
  closed_deals.sdr_id,
  closed_deals.sr_id,
  closed_deals.won_date,
  mql.first_contact_date,
  date_diff('day', mql.first_contact_date, closed_deals.won_date) as
  est_days_to_close
from closed_deals
inner join mql on closed_deals.mql_id = mql.mql_id

```

One row per closed deal (842 rows, before any join to the outcome side). `est_days_to_close` is computed, not synthesized, from real dates — but it's prefixed `est_` anyway here because it's a derived statistic rather than a passthrough of a source column, same convention as `business-case.md` applies everywhere else.

3.7 Marts: `fct_seller_ltv.sql` — the left join that has to go the right direction

```

-- models/marts/fct_seller_ltv.sql
with gmv as (
  -- the spine — every seller-month that actually produced GMV,
  attributed or not
  select * from {{ ref('int_olist__seller_monthly_gmv') }}
),

```

```

acquisition as (
  select * from {{ ref('int_olist__seller_acquisition') }}
)

select
  gmv.seller_id,
  gmv.month_start,
  gmv.gmv,
  gmv.total_freight,
  gmv.n_orders,
  gmv.avg_review_score,
  acquisition.origin,
  acquisition.lead_type,
  acquisition.business_type,
  acquisition.won_date,
  acquisition.est_days_to_close,
  -- Left join off the GMV spine, never the reverse: acquisition is only
  present for
  -- ~380 of the 3,095 sellers who ever generated GMV. Joining the other
  direction
  -- (acquisition as the spine) would silently drop the 88% of GMV that
  has no tracked
  -- channel from every total-marketplace number downstream – a much
  worse failure than
  -- just having a lot of `est_is_attributed = false` rows.
  (acquisition.seller_id is not null) as est_is_attributed
from gmv
left join acquisition on gmv.seller_id = acquisition.seller_id

```

3.8 Marts: `fct_channel_funnel.sql`

```

-- models/marts/fct_channel_funnel.sql
select
  origin,
  week_start,
  n_leads,
  n_closed,
  case when n_leads > 0 then n_closed::double / n_leads else null end as
  est_close_rate
from {{ ref('int_olist__funnel_weekly') }}

```

3.9 Tests

```

# models/marts/_marts__schema.yml
version: 2

models:
  - name: fct_seller_ltv

```

```

columns:
  - name: seller_id
    tests: [not_null]
  - name: month_start
    tests: [not_null]
  - name: gmv
    tests: [not_null]
  - name: est_days_to_close
    tests:
      - dbt_utils.accepted_range:
          min_value: 0
          row_condition: "est_is_attributed"
tests:
  - dbt_utils.unique_combination_of_columns:
      combination_of_columns: [seller_id, month_start]

```

```

-- tests/assert_attributed_sellers_within_bound.sql
-- Singular test: fails (returns a row) if the count of distinct attributed
sellers
-- in fct_seller_ltv ever exceeds 380 – the number verified by hand against
the raw
-- Kaggle files in business-case.md §3. A dbt refactor can legitimately
make this
-- number go DOWN (a stricter join), but it should never go UP – that would
mean the
-- join is inventing attribution that isn't in the source data.
select count(distinct seller_id) as n_attributed
from {{ ref('fct_seller_ltv') }}
where est_is_attributed
having count(distinct seller_id) > 380

```

Done when: `dbt build --target dev` (DuckDB) and `dbt build --target prod` (BigQuery) both pass from the same SQL — see Appendix C.6 in `tutorial.md` for the `profiles.yml` that makes this possible and the dialect table for where you'll actually hit divergence (mostly `SAFE_CAST` vs `TRY_CAST`, and `FILTER (WHERE ...)` needing to become `CASE WHEN` for BigQuery).

Be able to explain: why the acquisition join in §3.7 is a left join off `int_olist__seller_monthly_gmv` and not the reverse, and what `est_is_attributed = false` on 88% of sellers actually means for any channel-level conclusion drawn from this mart later.

Week 4 — Orchestration and infrastructure

4.1 Terraform: bucket and datasets

```

# infra/main.tf
terraform {
  required_providers {

```

```

    google = {
      source = "hashicorp/google"
      version = "~> 5.0"
    }
  }
}

provider "google" {
  project = var.project_id
  region  = var.region
}

variable "project_id" {
  type = string
}

variable "region" {
  type      = string
  default = "europe-west1"
}

variable "bucket_name" {
  type      = string
  default    = "profit-forecast-data" # kept from an earlier naming round
  description = "Existing or new GCS bucket name. Bucket names can't be
renamed in place."
}

resource "google_storage_bucket" "data" {
  name                = var.bucket_name
  location            = "EU"
  uniform_bucket_level_access = true
  force_destroy       = true # allows `terraform destroy` to
actually clean up –
                                # deliberate for a learning project;
remove for anything real
}

resource "google_bigquery_dataset" "raw" {
  dataset_id = "raw"
  location   = "EU"
}

resource "google_bigquery_dataset" "analytics" {
  dataset_id = "analytics"
  location   = "EU"
}

output "bucket_name" {
  value = google_storage_bucket.data.name
}

```

```

cd infra
terraform init
terraform plan -var="project_id=pnl-forecast"
terraform apply -var="project_id=pnl-forecast"
# ... later ...
terraform destroy -var="project_id=pnl-forecast"

```

`force_destroy = true` on the bucket is the detail most people miss and then can't `terraform destroy` cleanly — GCS refuses to delete a non-empty bucket by default.

4.2 Airflow: the weekly DAG

```

# dags/weekly_acquisition_roi.py
from __future__ import annotations

from datetime import datetime, timedelta

from airflow.decorators import dag, task
from airflow.exceptions import AirflowFailException

default_args = {
    "retries": 3,
    "retry_delay": timedelta(minutes=5),
    "retry_exponential_backoff": True,
}

@dag(
    schedule="@weekly",
    start_date=datetime(2026, 1, 1),
    catchup=False,
    default_args=default_args,
    tags=["acquisition-roi"],
)
def weekly_acquisition_roi():
    @task
    def ingest():
        import subprocess

        result = subprocess.run(
            ["python", "ingest.py", "--raw", "data/raw", "--out",
            "data/parquet",
            "--bucket", "profit-forecast-data"],
            capture_output=True, text=True,
        )
        if result.returncode != 0:
            raise AirflowFailException(f"ingest.py
failed:\n{result.stderr}")

    @task

```

```
def load():
    import subprocess

    result = subprocess.run(
        ["python", "load.py", "--bucket", "profit-forecast-data",
         "--project", "pnl-forecast"],
        capture_output=True, text=True,
    )
    if result.returncode != 0:
        raise AirflowFailException(f"load.py failed:\n{result.stderr}")

@task
def dbt_build():
    import subprocess

    result = subprocess.run(
        ["dbt", "build", "--target", "prod"],
        capture_output=True, text=True, cwd="pnl_forecast",
    )
    if result.returncode != 0:
        raise AirflowFailException(f"dbt build
failed:\n{result.stdout}\n{result.stderr}")

    ingest() >> load() >> dbt_build()

weekly_acquisition_roi()
```

Be able to explain: idempotency under mid-write retry. Walk through what happens if `load()` retries after `ingest()` succeeded but `load()` died halfway through a BigQuery load job:

- `ingest.py` used `gcloud storage rsync`, which only re-uploads changed files — safe to rerun.
- `load.py` used `WriteDisposition.WRITE_TRUNCATE` — a retried load job replaces the table wholesale rather than appending, so a half-loaded table from the failed attempt is simply overwritten, not duplicated into.
- `dbt build` models are `table` or `view` materializations by default — each run fully rebuilds from its `ref()`s, so a retry after a mid-run dbt failure just reruns the DAG from wherever it stopped; already-succeeded models aren't re-executed unless you pass `--full-refresh` on an incremental model (none of this project's models are incremental, deliberately, given the data volume fits comfortably in a full rebuild).

Every step in this pipeline is idempotent by construction, which is why `retries=3` in `default_args` is safe to set blindly — a retry can never corrupt state, only redo work.

Week 5 — Forecasting

5.1 `ltv_forecast.py`

```
#!/usr/bin/env python3
"""ltv_forecast.py – cohort-naive baseline + exponential-decay retention
curve,
rolling-origin backtested by cohort month, writing marts.fct_ltv_forecast
with a
model_version column.

Fit on the 380 sellers with a tracked acquisition record, over at most ~11
months of
post-close history each – a small n, stated here rather than hidden behind
a
confident-looking forecast. See business-case.md §E.

Usage:
    python ltv_forecast.py --duckdb data/local.duckdb --horizon 13
"""
from __future__ import annotations

import argparse
import logging

import duckdb
import numpy as np
import pandas as pd
from scipy.optimize import curve_fit

logger = logging.getLogger("ltv_forecast")

MODEL_VERSION = "exp_decay_v1"

def load_attributed_cohort_gmv(con: duckdb.DuckDBPyConnection) ->
pd.DataFrame:
    """One row per (seller_id, months_since_won) for attributed sellers
    only."""
    return con.execute("""
        select
            seller_id,
            date_diff('month', won_date, month_start) as months_since_won,
            gmv
        from fct_seller_ltv
        where est_is_attributed
            and date_diff('month', won_date, month_start) >= 0
        order by seller_id, months_since_won
    """).fetch_df()

def cohort_naive(cohort_medians: pd.Series, horizon: int) -> np.ndarray:
    """A seller's GMV at month t since close = the cohort's observed median
    GMV at t."""
    return np.array([
        cohort_medians.get(t, cohort_medians.iloc[-1]) if
```

```

len(cohort_medians) else 0.0)
    for t in range(horizon)
])

def _decay(t: np.ndarray, gmv0: float, half_life: float) -> np.ndarray:
    return gmv0 * np.power(0.5, t / half_life)

def fit_decay_curve(cohort_medians: pd.Series) -> tuple[float, float] |
None:
    if len(cohort_medians) < 3:
        return None
    t = cohort_medians.index.to_numpy(dtype=float)
    y = cohort_medians.to_numpy(dtype=float)
    try:
        (gmv0, half_life), _ = curve_fit(_decay, t, y, p0=[y[0] if y[0] > 0
else 1.0, 3.0],
maxfev=5000)
        return float(gmv0), float(half_life)
    except RuntimeError as exc:
        logger.warning("Decay curve failed to converge: %s", exc)
        return None

def wape(actual: np.ndarray, predicted: np.ndarray) -> float:
    return np.abs(actual - predicted).sum() / max(np.abs(actual).sum(), 1e-
9)

def rolling_origin_backtest(
    cohort_df: pd.DataFrame, horizon: int, n_folds: int = 3
) -> tuple[float, float]:
    """Backtests by cohort month depth, not calendar month – a cohort's
'month 3' is a
different calendar month for every seller depending on when they
closed."""
    max_month = int(cohort_df["months_since_won"].max())
    baseline_scores, model_scores = [], []

    for fold in range(n_folds):
        cutoff = max_month - fold - 1
        if cutoff < 2:
            continue
        train = cohort_df[cohort_df["months_since_won"] <= cutoff]
        test = cohort_df[cohort_df["months_since_won"] == cutoff + 1]
        if test.empty:
            continue

        train_medians = train.groupby("months_since_won")["gmv"].median()
        baseline_pred = cohort_naive(train_medians, cutoff + 2)[-1]
        actual = test["gmv"].median()
        baseline_scores.append(wape(np.array([actual]),
np.array([baseline_pred])))

```

```

        fit = fit_decay_curve(train_medians)
        if fit is None:
            model_scores.append(baseline_scores[-1])
            continue
        gmvo, half_life = fit
        model_pred = _decay(np.array([cutoff + 1]), gmvo, half_life)[0]
        model_scores.append(wape(np.array([actual]),
np.array([model_pred])))

    if not baseline_scores:
        return float("nan"), float("nan")
    return float(np.mean(baseline_scores)), float(np.mean(model_scores))

def forecast_cohort(cohort_df: pd.DataFrame, horizon: int) -> pd.DataFrame:
    cohort_medians = cohort_df.groupby("months_since_won")["gmvo"].median()
    baseline_wape, model_wape = rolling_origin_backtest(cohort_df, horizon)

    fit = fit_decay_curve(cohort_medians)
    if fit is not None and (np.isnan(model_wape) or model_wape <
baseline_wape):
        gmvo, half_life = fit
        forecast = _decay(np.arange(horizon, dtype=float), gmvo, half_life)
        model_used = MODEL_VERSION
        headline_wape = model_wape
    else:
        # Guardrail from tutorial.md: if the decay curve loses to cohort-
naive,
        # ship cohort-naive and say so – explicitly, in the output, not
silently.
        forecast = cohort_naive(cohort_medians, horizon)
        model_used = "cohort_naive"
        headline_wape = baseline_wape
        logger.info("cohort_naive beat exp_decay (%.3f vs %.3f WAPE),
shipping baseline",
                    baseline_wape, model_wape if not np.isnan(model_wape)
else float("inf"))

    return pd.DataFrame({
        "months_since_won": range(horizon),
        "est_gmv_forecast": forecast,
        "model_version": model_used,
        "backtest_wape": headline_wape,
        "n_attributed_sellers_fit": cohort_df["seller_id"].nunique(),
    })

def main(argv: list[str] | None = None) -> int:
    p = argparse.ArgumentParser()
    p.add_argument("--duckdb", required=True)
    p.add_argument("--horizon", type=int, default=13)
    p.add_argument("-v", "--verbose", action="store_true")
    args = p.parse_args(argv)

```

```

logging.basicConfig(level=logging.DEBUG if args.verbose else
logging.INFO,
                    format="%(asctime)s %(levelname)-8s %(message)s")

con = duckdb.connect(args.duckdb)
cohort_df = load_attributed_cohort_gmv(con)

n_sellers = cohort_df["seller_id"].nunique()
if n_sellers < 30:
    logger.warning(
        "Fitting a decay curve on %d attributed sellers – treat this
forecast as "
        "directional, not precise. See business-case.md §E.",
n_sellers,
    )

results = forecast_cohort(cohort_df, args.horizon)

con.execute("CREATE SCHEMA IF NOT EXISTS marts")
con.execute("CREATE OR REPLACE TABLE marts.fct_ltv_forecast AS SELECT *
FROM results")
logger.info("Wrote %d forecast rows to marts.fct_ltv_forecast
(model=%s, n_sellers=%d)",
            len(results), results["model_version"].iloc[0], n_sellers)
return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

Be able to explain: why WAPE, not MAPE. MAPE divides each error by that period's actual value — a seller's first month with 20 BRL of GMV and a forecast of 60 BRL contributes a 200% error that dominates the average, even though the absolute miss (40 BRL) is trivial next to a mature seller's monthly GMV. WAPE ($\text{sum}(|\text{error}|) / \text{sum}(|\text{actual}|)$) weights every period's error by its share of total volume instead of treating every period equally regardless of scale — the right choice whenever near-zero actuals are common, which a seller's ramp-up month or a churned seller's tail both guarantee here.

Week 6 — Scenarios and sensitivity

6.1 `channel_cost_benchmarks.csv` (seed)

Fully synthesized — Olist doesn't publish CAC. Every row needs a `source` describing where the range came from; "`placeholder – replace with a cited benchmark`" is an honest placeholder, a silent guess is not.

```

origin,est_cost_per_lead,est_sdr_hours_per_close,source
organic_search,8.00,4.0,placeholder – replace with a cited SEO/content-cost
benchmark
paid_search,45.00,3.5,placeholder – replace with a cited CPC benchmark for

```

```

the vertical
social,22.00,4.5,placeholder – replace with a cited paid-social CAC
benchmark
direct_traffic,2.00,3.0,placeholder – near-zero acquisition cost, mostly
brand/referral-driven
email,5.00,2.5,placeholder – replace with a cited email-marketing cost-per-
lead figure
referral,10.00,2.0,placeholder – replace with a cited referral-program cost
figure
display,30.00,5.0,placeholder – replace with a cited display-ad CAC
benchmark
other,15.00,4.0,placeholder – catch-all, wide uncertainty band deliberately
other_publicities,18.00,4.0,placeholder – replace with a cited offline/PR
cost figure
unknown,15.00,4.0,placeholder – same as 'other'; unattributed origin gets
no better a number

```

6.2 `scenario_parameters.csv` (seed)

```

scenario_name,close_rate_multiplier,take_rate,cac_multiplier,sdr_hours_mult
iplier,retention_half_life_months,discount_rate
base,1.00,0.15,1.00,1.00,4.0,0.10
pessimistic,0.80,0.12,1.30,1.20,2.5,0.15
optimistic,1.20,0.18,0.80,0.85,6.0,0.08
sdr_capacity_crunch,0.70,0.15,1.00,1.50,4.0,0.10
cac_inflation,1.00,0.15,1.60,1.10,4.0,0.10
retention_shock,1.00,0.15,1.00,1.00,1.5,0.10

```

`take_rate` and `retention_half_life_months` are declared, not measured — Olist's actual commission rate isn't published, and the half-life is a modeling choice standing in for a real churn curve this project doesn't have the cohort-depth to fit precisely (§5).

6.3 `fct_roi_scenario_grid.sql` — the full formula from `business-case.md` §C

```

-- models/marts/fct_roi_scenario_grid.sql
with funnel as (
    select * from {{ ref('fct_channel_funnel') }}
),

scenarios as (
    select * from {{ ref('scenario_parameters') }}
),

cost_benchmarks as (
    select * from {{ ref('channel_cost_benchmarks') }}
),

ltv_forecast as (
    select * from {{ source('marts', 'fct_ltv_forecast') }} -- written by

```

```

ltv_forecast.py
),

-- one row per channel, historical close rate observed to date
channel_close_rate as (
    select
        origin,
        sum(n_closed)::double / nullif(sum(n_leads), 0) as est_close_rate
    from funnel
    group by 1
),

grid as (
    select
        funnel.origin,
        funnel.week_start,
        scenarios.scenario_name,
        funnel.n_leads,
        channel_close_rate.est_close_rate * scenarios.close_rate_multiplier
as est_close_rate,
        cost_benchmarks.est_cost_per_lead * scenarios.cac_multiplier as
est_cost_per_lead,
        cost_benchmarks.est_sdr_hours_per_close *
scenarios.sdr_hours_multiplier
        as est_sdr_hours_per_close,
        scenarios.take_rate,
        scenarios.retention_half_life_months,
        scenarios.discount_rate
    from funnel
    inner join channel_close_rate on funnel.origin =
channel_close_rate.origin
    cross join scenarios
    inner join cost_benchmarks on funnel.origin = cost_benchmarks.origin
),

sellers_won as (
    select
        *,
        n_leads * est_close_rate as est_sellers_won
    from grid
),

-- cross join the won-seller cohort onto the forecast horizon to get a
monthly revenue
-- stream per channel/scenario/week-cohort, then roll it up into
CAC/LTV/ROI
sdr_hourly_cost as (
    select 35.00 as est_sdr_cost_per_hour -- placeholder – replace with a
cited fully-loaded SDR cost
),

priced as (
    select
        sellers_won.*,

```

```

        sdr_hourly_cost.est_sdr_cost_per_hour,
        -- CAC is paid in full at close, independent of any future revenue
        sellers_won.est_sellers_won * (
            sellers_won.est_cost_per_lead /
nullif(sellers_won.est_close_rate, 0)
            + sdr_hourly_cost.est_sdr_cost_per_hour *
sellers_won.est_sdr_hours_per_close
        ) as est_cac_total,
        -- LTV: sum of forecasted monthly take-revenue, discounted, over
the retention curve
        sellers_won.est_sellers_won * (
            select sum(
                ltv_forecast.est_gmv_forecast
                * sellers_won.take_rate
                * power(0.5, ltv_forecast.months_since_won /
sellers_won.retention_half_life_months)
                / power(1 + sellers_won.discount_rate / 12,
ltv_forecast.months_since_won)
            )
            from ltv_forecast
        ) as est_ltv_total
    from sellers_won
    cross join sdr_hourly_cost
)

select
    origin,
    week_start,
    scenario_name,
    est_sellers_won,
    est_cac_total,
    est_ltv_total,
    (est_ltv_total - est_cac_total) / nullif(est_cac_total, 0) as est_roi
from priced

```

`business-case.md` \$D expects `retention_half_life_months` and `est_cost_per_lead` to dominate the tornado chart — both synthesized, both wider-ranged in the scenario seed than the historically observed `close_rate`, deliberately.

6.4 `tornado.py` — one-at-a-time sensitivity

```

#!/usr/bin/env python3
"""tornado.py — OAT sensitivity: vary each scenario parameter across its
plausible
range, hold others at base, rank by ROI swing.

Usage:
    python tornado.py --duckdb data/local.duckdb --out tornado.png
"""
from __future__ import annotations

```

```

import argparse

import duckdb
import matplotlib.pyplot as plt
import pandas as pd

PARAM_RANGES = {
    # param: (low, base, high) – pulled from the min/max across
    scenario_parameters.csv rows
    "close_rate_multiplier": (0.70, 1.00, 1.20),
    "take_rate": (0.12, 0.15, 0.18),
    "cac_multiplier": (0.80, 1.00, 1.60),
    "sdr_hours_multiplier": (0.85, 1.00, 1.50),
    "retention_half_life_months": (1.5, 4.0, 6.0),
    "discount_rate": (0.08, 0.10, 0.15),
}

BASE_PARAMS = {
    "close_rate_multiplier": 1.00,
    "cac_multiplier": 1.00,
    "sdr_hours_multiplier": 1.00,
    "take_rate": 0.15,
    "retention_half_life_months": 4.0,
    "discount_rate": 0.10,
}

def roi_at(con: duckdb.DuckDBPyConnection, overrides: dict[str, float]) ->
float:
    """Portfolio ROI with one parameter overridden – re-executes the exact
    formula
    shape from fct_roi_scenario_grid.sql (§6.3) against the already-
    materialized
    fct_channel_funnel / channel_cost_benchmarks / fct_ltv_forecast tables,
    so this
    can never silently drift from the dbt model. `overrides` values aren't
    limited to
    what's in scenario_parameters.csv – that's the whole point of OAT: it
    probes the
    plausible range, not just the six named scenarios.
    """
    p = {**BASE_PARAMS, **overrides}
    query = f"""
        with channel_close_rate as (
            select origin, sum(n_closed)::double / nullif(sum(n_leads), 0)
as est_close_rate
            from fct_channel_funnel
            group by 1
        ),
        grid as (
            select
                f.origin,
                f.n_leads,
    """

```

```

        channel_close_rate.est_close_rate *
        {p['close_rate_multiplier']} as est_close_rate,
        c.est_cost_per_lead * {p['cac_multiplier']} as
est_cost_per_lead,
        c.est_sdr_hours_per_close * {p['sdr_hours_multiplier']} as
est_sdr_hours_per_close
    from fct_channel_funnel f
    inner join channel_close_rate using (origin)
    inner join channel_cost_benchmarks c using (origin)
),
sellers_won as (
    select *, n_leads * est_close_rate as est_sellers_won from grid
),
priced as (
    select
        sellers_won.est_sellers_won * (
            sellers_won.est_cost_per_lead /
nullif(sellers_won.est_close_rate, 0)
            + 35.00 * sellers_won.est_sdr_hours_per_close --
placeholder SDR cost/hour, see §6.3
        ) as est_cac_total,
        sellers_won.est_sellers_won * (
            select sum(
                l.est_gmv_forecast * {p['take_rate']}
                * power(0.5, l.months_since_won /
{p['retention_half_life_months']})
                / power(1 + {p['discount_rate']} / 12,
l.months_since_won)
            )
            from marts.fct_ltv_forecast l
        ) as est_ltv_total
    from sellers_won
)
    select sum(est_ltv_total - est_cac_total) /
nullif(sum(est_cac_total), 0)
    from priced
"""
    return con.execute(query).fetchone()[0]

def main() -> int:
    p = argparse.ArgumentParser()
    p.add_argument("--duckdb", required=True)
    p.add_argument("--out", default="tornado.png")
    args = p.parse_args()

    con = duckdb.connect(args.duckdb)
    base_roi = roi_at(con, {})

    swings = []
    for param, (low, base, high) in PARAM_RANGES.items():
        low_roi = roi_at(con, {param: low})
        high_roi = roi_at(con, {param: high})
        swings.append({

```

```

        "param": param,
        "low_delta": low_roi - base_roi,
        "high_delta": high_roi - base_roi,
        "range": abs(high_roi - low_roi),
    })

df = pd.DataFrame(swings).sort_values("range")

fig, ax = plt.subplots(figsize=(8, 5))
ax.barh(df["param"], df["high_delta"] - df["low_delta"],
left=df["low_delta"])
ax.axvline(0, color="black", linewidth=0.8)
ax.set_xlabel("ROI swing vs base scenario")
ax.set_title("Tornado chart – OAT sensitivity")
fig.tight_layout()
fig.savefig(args.out, dpi=150)
print(df.to_string(index=False))
return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

Expect (per [business-case.md](#) §D) `retention_half_life_months` and `est_cost_per_lead` to rank above `close_rate_multiplier` — that's the honest finding worth stating in your README rather than a bug to chase: the model's biggest lever is the parameter you know least about.

6.5 `simulate.py` — Monte Carlo

```

#!/usr/bin/env python3
"""simulate.py – Monte Carlo over scenario parameters, triangular
distributions,
10k draws, writes marts.fct_roi_dist with P10/P50/P90.

Usage:
    python simulate.py --duckdb data/local.duckdb --draws 10000
"""
from __future__ import annotations

import argparse
import logging

import duckdb
import numpy as np
import pandas as pd

logger = logging.getLogger("simulate")

# param: (min, mode, max) – same ranges as tornado.py's PARAM_RANGES,
reinterpreted as
# a triangular distribution's three parameters

```

```

TRIANGULAR_PARAMS = {
    "close_rate_multiplier": (0.70, 1.00, 1.20),
    "take_rate": (0.12, 0.15, 0.18),
    "cac_multiplier": (0.80, 1.00, 1.60),
    "sdr_hours_multiplier": (0.85, 1.00, 1.50),
    "retention_half_life_months": (1.5, 4.0, 6.0),
    "discount_rate": (0.08, 0.10, 0.15),
}

def sample_params(n_draws: int, rng: np.random.Generator) -> pd.DataFrame:
    return pd.DataFrame({
        param: rng.triangular(low, mode, high, size=n_draws)
        for param, (low, mode, high) in TRIANGULAR_PARAMS.items()
    })

def run_simulation(con: duckdb.DuckDBPyConnection, draws: pd.DataFrame) ->
pd.DataFrame:
    funnel = con.execute("""
        select origin, sum(n_leads) as n_leads,
               sum(n_closed)::double / nullif(sum(n_leads), 0) as
base_close_rate
        from fct_channel_funnel
        group by 1
    """).fetch_df()

    cost = con.execute("select * from
channel_cost_benchmarks").fetch_df().set_index("origin")
    ltv_curve = con.execute("""
        select months_since_won, est_gmv_forecast from
marts.fct_ltv_forecast
    """).fetch_df()

    results = []
    for i, row in draws.iterrows():
        sellers_won = funnel["n_leads"] * funnel["base_close_rate"] *
row["close_rate_multiplier"]
        cost_per_lead = funnel["origin"].map(cost["est_cost_per_lead"]) *
row["cac_multiplier"]
        sdr_hours = funnel["origin"].map(cost["est_sdr_hours_per_close"]) *
row["sdr_hours_multiplier"]
        cac = sellers_won * (cost_per_lead /
funnel["base_close_rate"].clip(lower=1e-6)
                             + 35.00 * sdr_hours) # placeholder SDR
cost/hour, see fct_roi_scenario_grid.sql

        discount_factors = 1 / np.power(
            1 + row["discount_rate"] / 12,
            ltv_curve["months_since_won"].to_numpy()
        )
        retention = np.power(0.5, ltv_curve["months_since_won"].to_numpy()
/ row["retention_half_life_months"])
        ltv_per_seller = (ltv_curve["est_gmv_forecast"].to_numpy() *

```

```

row["take_rate"]
        * retention * discount_factors).sum()
    ltv = sellers_won * ltv_per_seller

    roi = (ltv.sum() - cac.sum()) / max(cac.sum(), 1e-9)
    results.append(roi)

    if i % 1000 == 0:
        logger.debug("Draw %d/%d", i, len(draws))

    return pd.DataFrame({"draw": range(len(results)), "est_portfolio_roi":
results})

def main() -> int:
    p = argparse.ArgumentParser()
    p.add_argument("--duckdb", required=True)
    p.add_argument("--draws", type=int, default=10_000)
    p.add_argument("--seed", type=int, default=42)
    p.add_argument("-v", "--verbose", action="store_true")
    args = p.parse_args()
    logging.basicConfig(level=logging.DEBUG if args.verbose else
logging.INFO,
                        format="%(asctime)s %(levelname)-8s %(message)s")

    rng = np.random.default_rng(args.seed)
    con = duckdb.connect(args.duckdb)

    draws = sample_params(args.draws, rng)
    sim = run_simulation(con, draws)

    p10, p50, p90 = sim["est_portfolio_roi"].quantile([0.10, 0.50, 0.90])
    logger.info("P10=%.2f P50=%.2f P90=%.2f", p10, p50, p90)

    con.execute("CREATE SCHEMA IF NOT EXISTS marts")
    con.execute("CREATE OR REPLACE TABLE marts.fct_roi_dist AS SELECT *
FROM sim")
    logger.info("Wrote %d draws to marts.fct_roi_dist", len(sim))
    return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

Why `np.random.default_rng(seed)` rather than the legacy `np.random.seed(...)` global state: it's a local generator instance, so running this alongside other code that also uses numpy's RNG can't cross-contaminate reproducibility — the seed you set here only ever affects `rng`.

Run it, then think, per [tutorial.md](#)'s instruction: 13 weeks × ~10 channels × 6 scenarios is a few hundred rows — trivial for BigQuery regardless of partitioning. Now imagine the same cross join at seller × week grain instead of channel × week: 3,095 sellers × 13 weeks × 6 scenarios is two orders of magnitude more rows from the scenario fan-out alone. That contrast — where a design choice about grain changes a job from

"instant" to "needs partitioning discipline" — is the lesson this step is built to teach, same as it was for the previous version of this project.

Be able to explain: why CAC being paid in full at close, while revenue is a conditional, decaying stream, makes the downside tail worse than a naive symmetric-uncertainty view would predict. `est_cac_total` in `fct_roi_scenario_grid.sql` is computed the moment a deal closes, independent of anything that happens afterward. `est_ltv_total` only accrues if the seller actually lists product, gets orders, and doesn't churn — and it decays under the retention curve the whole time. When `retention_half_life_months` draws low in the same simulation run as `close_rate_multiplier` draws high (an `sdr_capacity_crunch`-adjacent draw, or a "closed a lot of sellers who barely shipped" draw), CAC is spent against a cohort that mostly evaporates before paying it back. Nothing in the formula lets a strong close rate "cancel out" weak retention — it can only ever subtract. That's a structural asymmetry, not a modeling artifact, and it's why the P10 tail is worse than a naive model would predict.

As with `tutorial.md`: the SQL above assumes the exact source-column names Kaggle's exports use today. Run `describe select * from read_csv_auto('olist_closed_deals_dataset.csv') limit 0` (DuckDB) the moment you have the real files, and check null rates on `declared_monthly_revenue`, `declared_product_catalog_size`, `has_company`, `has_gtin`, and `average_stock` before building anything that assumes those columns are populated — in the version of this dataset this project was written against, most of them mostly aren't.